

TECHIN

JavaScript

Masyvai

JavaScript masyvai

- **JavaScript‘e masyvus galima sukurti dviem būdais.**

Pirmas:

```
let arr = new Array();
```

tuščias masyvas [ ]

```
// Create an empty array with length 5
const arr = new Array(5);

console.log(arr);        // [ <5 empty items> ]
console.log(arr.length); // 5
```

tuščias masyvas [ ] su ilgiu 5

```
const numbers = new Array(5).fill(0);

console.log(numbers); // [0, 0, 0, 0, 0]
```

masyvo užpildymas reikšmėmis

JavaScript masyvai

- **JavaScript‘e masyvus galima sukurti dviem būdais.**

Antras:

```
let arr = [];
```

Dažniausiai naudojama antra sintaksė.
Galima iš karto kurti su reikšmėmis, kurios vadinamos masyvo elementais:

```
let fruits = ["Apple", "Orange", "Plum"];
```

Masyvo elemento pakeitimas:

```
1  fruits[2] = 'Pear'; // now ["Apple", "Orange", "Pear"]
```

Masyvo elemento pridėjimas:

```
1  fruits[3] = 'Lemon'; // now ["Apple", "Orange", "Pear", "Lemon"]
```



JavaScript masyvai

- **Sukūrus masyvą, atmintyje išskiriama vieta, kur saugomas masyvas. Kintamojo vardas tik nuoroda (reference) į tą atminties vietą.**
- **Jeigu vieną masyvo tipo kintamąjį priskirsime kitam, tai susikurs du kintamieji, rodantys į tą patį masyvą.**
- **Jeigu pakeisime vieno kintamojo reikšmę, tai pasikeis ir kito kintamojo reikšmė.**

```
1  let fruits = ["Banana"]
2
3  let arr = fruits; // copy by reference (two variables reference the same array)
4
5  alert( arr === fruits ); // true
6
7  arr.push("Pear"); // modify the array by reference
8
9  alert( fruits ); // Banana, Pear - 2 items now
```



JavaScript masyvai
Spread operator (...)

- **Spread operator (...) – išskaido elementus iš masyvo.**

```
const arrValue = ['My', 'name', 'is', 'Jack'];

console.log(arrValue);   // ["My", "name", "is", "Jack"]
console.log(...arrValue); // My name is Jack
```

```
const arr1 = ['one', 'two'];
const arr2 = [...arr1, 'three', 'four', 'five'];

console.log(arr2);
//  Output:
//  ["one", "two", "three", "four", "five"]
```

JavaScript masyvai

Masyvų dubliavimas/kopijavimas

```
const sheeps = ['🐑', '🐑', '🐑'];

// Old way
const cloneSheeps = sheeps.slice();

// ES6 way
const cloneSheepsES6 = [...sheeps];
```

```
const numbersOne = [1, 2, 3];
const numbersTwo = [4, 5, 6];
const numbersCombined = [...numbersOne, ...numbersTwo];
```



JavaScript masyvai

Rest parameter

- Kai spread operatorius naudojamas funkcijai paduoti duomenis, jis vadinamas rest parameter.
- Funkcijos viduje atskiri parametrai sujungiami į masyvą.

```
let func = function(...args) {
    console.log(args);
}

func(3); // [3]
func(4, 5, 6); // [4, 5, 6]
```



JavaScript masyvai

Masyvo ilgis:

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits.length ); // 3
```

Viso masyvo pasiekimas:

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits ); // Apple,Orange,Plum
```

Masyvas gali saugoti įvairių tipų duomenis:

```
// mix of values
let arr = [ 'Apple', { name: 'John' }, true, function() { alert('hello'); } ];

// get the object at index 1 and then show its name
alert( arr[1].name ); // John

// get the function at index 3 and run it
arr[3](); // hello
```



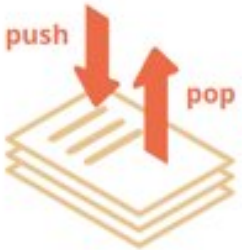
JavaScript masyvų metodai

pop( ) – panaikina paskutinį masyvo elementą

```
1  let fruits = ["Apple", "Orange", "Pear"];
2
3  alert( fruits.pop() ); // remove "Pear" and alert it
4
5  alert( fruits ); // Apple, Orange
```

push( ) – prideda elementą į masyvo galą

```
1  let fruits = ["Apple", "Orange"];
2
3  fruits.push("Pear");
4
5  alert( fruits ); // Apple, Orange, Pear
```

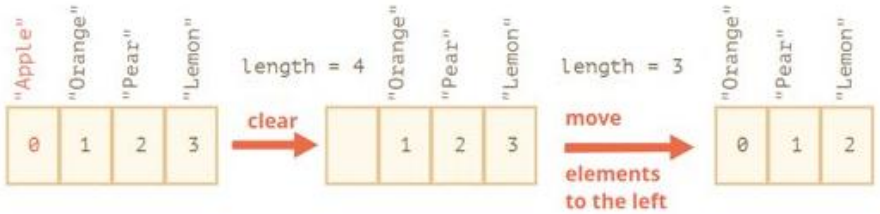


Veiksmai atliekami su tuo pačiu masyvu, nekuriant naujo

JavaScript masyvų metodai

shift( ) – panaikina pirmą masyvo elementą, visą masyvą pastumia į priekį, antras elementas tampa pirmas.

```
1  let fruits = ["Apple", "Orange", "Pear"];
2
3  alert( fruits.shift() ); // remove Apple and alert it
4
5  alert( fruits ); // Orange, Pear
```



unshift( ) – prideda elementą į masyvo pradžią

```
1  let fruits = ["Orange", "Pear"];
2
3  fruits.unshift('Apple');
4
5  alert( fruits ); // Apple, Orange, Pear
```

JavaScript masyvų metodai

splice( ) – gali įterpti, ištrinti, pakeisti masyvo elementus. Nekuria naujo masyvo, redaguoja senąjį.

Bendra sintaksė:

```
arr.splice(index[, deleteCount, elem1, ..., elemN])
```

index – nurodomas elementas nuo kurio pradėti trinti.

deleteCount – nurodo, kiek elementų trinti.

elem1, ..., elemN – nurodo elementus, kurie bus įterpti vietoje ištrintų

Metodo grąžinama reikšmė – ištrintų elementų masyvas.

Ištrinti elementą:

```
let arr = ["I", "study", "JavaScript"];

arr.splice(1, 1); // from index 1 remove 1 element

alert( arr ); // ["I", "JavaScript"]
```

JavaScript masyvų metodai
splice( )

Ištrina kelis elementus ir pakeičia juos naujais:

```
let arr = ["I", "study", "JavaScript", "right", "now"];

// remove 3 first elements and replace them with another
arr.splice(0, 3, "Let's", "dance");

alert( arr ) // now ["Let's", "dance", "right", "now"]
```

Įterpia elementą:

```
let arr = ["I", "study", "JavaScript"];

// from index 2
// delete 0
// then insert "complex" and "language"
arr.splice(2, 0, "complex", "language");

alert( arr ); // "I", "study", "complex", "language", "JavaScript"
```



JavaScript masyvų metodai
splice( )

Ištrina ir grąžina ištrinų elementų masyvą:

```
let arr = ["I", "study", "JavaScript", "right", "now"];

// remove 2 first elements
let removed = arr.splice(0, 2);

alert( removed ); // "I", "study" <-- array of removed elements
```



JavaScript masyvų metodai
slice( )

Nukopijuoja dalį masyvo į naują masyvą, nepakeisdamas senojo. Grąžina naujai sukurtą masyvą.

Bendra sintaksė: (start – nuo kurios pozicijos kopijuoti, end – iki kurios)

```
arr.slice([start], [end])
```

```
let arr = ["t", "e", "s", "t"];

alert( arr.slice(1, 3) ); // e,s (copy from 1 to 3)

alert( arr.slice(-2) ); // s,t (copy from -2 till the end)
```

Jeigu naudosime be argumentų, arr.slice() padarys viso masyvo kopiją. Dažnai naudojama, kai norima keisti masyvą nedarant įtakos originalui.



JavaScript masyvų metodai

map( )

Sukuria naują masyvą, su masyvo elementais atlikdamas veiksmus, aprašytus funkcija (atlieka elementų transformavimą).

Pvz., sukuria naują masyvą, padaugindamas kiekvieną elementą iš dviejų:

```
var numbers1 = [45, 4, 9, 16, 25];
var numbers2 = numbers1.map(myFunction);

function myFunction(value, index, array) {
  return value * 2;
}
```

Funkcija myFunction yra vadinama callback function. Ji vykdoma sinchroniškai.



JavaScript masyvų metodai
forEach( )

Atlieka funkcijoje aprašytus veiksmus su kiekvienu masyvo elementu tame pačiame masyve nekuriant naujo. Nieko negrąžina, return ignoruojamas.

Kadangi nėra return, reikia su funkcijos rezultatais kažką daryti (pvz., spausdinti), nes funkcijai įvykus, jos skaičiavimai bus ištrinti:

```
["Bilbo", "Gandalf", "Nazgul"].forEach((item, index, array) =>
  alert(`${item} is at index ${index} in ${array}`);
});
```

Arba rankiniu būdu priskiriame pačių sukurtam naujam masyvui:

```
const apps = ['WhatsApp', 'Instagram', 'Facebook'];
const playStore = [];

apps.forEach(function(item){
  playStore.push(item)
});

console.log(playStore);
```

JavaScript masyvų metodai
concat( )

- Sujungia kelis masyvus.
- Taip pat gali prijungti atskirus elementus prie masyvo.
- Sukuriamas naujas masyvas.

```
let arr = [1, 2];

// create an array from: arr and [3,4]
alert( arr.concat([3, 4]) ); // 1,2,3,4

// create an array from: arr and [3,4] and [5,6]
alert( arr.concat([3, 4], [5, 6]) ); // 1,2,3,4,5,6

// create an array from: arr and [3,4], then add values 5 and 6
alert( arr.concat([3, 4], 5, 6) ); // 1,2,3,4,5,6
```



JavaScript masyvų metodai
filter( )

- **Sukuria naują masyvą iš funkcijoje aprašytą sąlygą atitinkančių elementų.**

```
let results = arr.filter(function(item, index, array) {
  // if true item is pushed to results and the iteration continues
  // returns empty array if nothing found
});
```

```
let users = [
  {id: 1, name: "John"},
  {id: 2, name: "Pete"},
  {id: 3, name: "Mary"}
];

// returns array of the first two users
let someUsers = users.filter(item => item.id < 3);

alert(someUsers.length); // 2
```



JavaScript masyvų metodai
find( )

- **Grąžina masyvo elementą, kuris pirmas atitinka paieškos funkcijoje nurodytą sąlygą.**

```
1  const array1 = [5, 12, 8, 130, 44];
2
3  const found = array1.find(element => element > 10);
4
5  console.log(found);
6  // expected output: 12
7
```

- **Paieškos funkcija callback turi vieną argumentą – masyvo elementą. Taip pat dar gali turėti elemento indeksą ir patį masyvą:**

(currentValue, index, arr) => {...}

Pvz.

```
1  const inventory = [
2    {name: 'apples', quantity: 2},
3    {name: 'bananas', quantity: 0},
4    {name: 'cherries', quantity: 5}
5  ];
6
7  function isCherries(fruit) {
8    return fruit.name === 'cherries';
9  }
10
11 console.log(inventory.find(isCherries));
12 // { name: 'cherries', quantity: 5 }
```

JavaScript masyvų metodai
sort( )

- Tai rikiavimo metodas, kuris nekuria naujo masyvo, o pakeitimus atlieka originaliame masyve.
- Skirtingai rikiuojame string tipo duomenis ir skaičius.
- Jei masyve laikomi string tipo duomenys:

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];
fruits.sort();          // Surikiuoja pagal alfabetą A-Z
fruits.reverse();       // Surikiuoja atvirkščia tvarka Z-A
```

JavaScript masyvų metodai
sort( )

- **Jei masyve laikomi skaičiai, rikiavimas pagal abėcėlę netiks. Tuomet reikia panaudoti callback funkciją, kad rikiuotų pagal skaitinę reikšmę.**

Surikiuoja pagal didėjimo tvarka, ascending

```
var points = [40, 100, 1, 5, 25, 10];
points.sort(function(a,b){return a-b});
console.log(points);
```

Surikiuoja mažėjimo tvarka descending

```
var points = [40, 100, 1, 5, 25, 10];
points.sort(function(a,b){return a-b}).reverse();
console.log(points);
```

arba

```
function myFunction( arr ) {
  return arr.sort((a, b) => b - a)
}
```



JavaScript masyvų metodai

sort( )

```
var books = [

  { title: 'CKnyga #1', price: 10.25 },

  { title: 'AKnyga #2', price: 5.15 },

  { title: 'GKnyga #3', price: 7.32 },

  { title: 'BKnyga #4', price: 54.01 },

  { title: 'MKnyga #5', price: 77.17 },

];

books.sort(function(a,b){
if (a.price>b.price) return -1;
if (a.price<b.price) return 1;
  return 0;
});
console.log(books);
```

```
▼(5) [{…}, {…}, {…}, {…}, {…}]
  ▶0: {title: "MKnyga #5", price: 77.17}
  ▶1: {title: "BKnyga #4", price: 54.01}
  ▶2: {title: "CKnyga #1", price: 10.25}
  ▶3: {title: "GKnyga #3", price: 7.32}
  ▶4: {title: "AKnyga #2", price: 5.15}
    length: 5
  ▶__proto__: Array(0)
```

```
var books = [

  { title: 'CKnyga #1', price: 10.25 },

  { title: 'AKnyga #2', price: 5.15 },

  { title: 'GKnyga #3', price: 7.32 },

  { title: 'BKnyga #4', price: 54.01 },

  { title: 'MKnyga #5', price: 77.17 },

];

books.sort(function(a,b){
if (a.price>b.price) return 1;
if (a.price<b.price) return -1;
  return 0;
});
console.log(books);
```

```
▼(5) [{…}, {…}, {…}, {…}, {…}]
  ▶0: {title: "AKnyga #2", price: 5.15}
  ▶1: {title: "GKnyga #3", price: 7.32}
  ▶2: {title: "CKnyga #1", price: 10.25}
  ▶3: {title: "BKnyga #4", price: 54.01}
  ▶4: {title: "MKnyga #5", price: 77.17}
    length: 5
  ▶__proto__: Array(0)
```



JavaScript masyvų metodai

sort( )

```
let employees = [
    {
        firstName: 'John',
        lastName: 'Doe',
        age: 27,
        joinedDate: 'December 15, 2017'
    },

    {
        firstName: 'Ana',
        lastName: 'Rosy',
        age: 25,
        joinedDate: 'January 15, 2019'
    },

    {
        firstName: 'Zion',
        lastName: 'Albert',
        age: 30,
        joinedDate: 'February 15, 2011'
    }
];
```

Pagal vardus (string)

```
employees.sort((a, b) => {
    let fa = a.firstName.toLowerCase(),
        fb = b.firstName.toLowerCase();

    if (fa < fb) {
        return -1;
    }
    if (fa > fb) {
        return 1;
    }
    return 0;
});
```

Išvedimas

```
employees.forEach((e) => {
    console.log(`${e.firstName} ${e.lastName}`);
});
```

Pagal datas (date)

```
employees.sort((a, b) => {
    let da = new Date(a.joinedDate),
        db = new Date(b.joinedDate);
    return da - db;
});
```

Išvedimas

```
employees.forEach((e) => {
    console.log(`${e.firstName} ${e.lastName} ${e.joinedDate}`);
});
```

JavaScript masyvų metodai
reduce( )

- Šis metodas iteruoja masyvą nuo pradžios iki galo, t.y. ima po vieną masyvo reikšmę ir funkcijos pagalba atlieka skaičiavimus, tokius, kad pagal tam tikras taisykles masyvo reikšmės tampa viena reikšme. Pavyzdžiui susumuoja visas masyvo reikšmes.

```
//Implementation One
let answer1 = array.reduce(callback, initialValue);

//Implementation Two
let answer2 = array.reduce(callback);
```

- reduce metodas priima du parametrus:

callback - tai funkcija, kuri atlieka skaičiavimus ir grąžina rezultatą, vadinama reducer.

initialValue – tai pradinė reikšmė, nuo kurios pradėsime skaičiavimus. Nebūtina nurodyti. Kai ji nenurodyta pradinė reikšmė yra masyvo pirmas elementas arr[0].



JavaScript masyvų metodai
reduce( )

```
function callbackFn(previousValue, currentValue, currentIndex, array) { /**/}
```

callback (reducer) priima keturis parametrus:

1. **previousValue – tai reikšmė gauta iš ankstesnio callback kvietimo. Pirmą prasisukimą ši reikšmė lygi initialValue. Jei initialValue nenurodyta, tai masyvo pirmas elementas.**
2. **currentValue – tai einamasis masyvo elementas. Pirmu prasisukimu, jei nurodyta innitialValue, tai currentValue yra arr[0], priešingu atveju arr[1].**
3. **currentIndex – tai einamojo masyvo elemento indeksas. Pirmu prasisukimu currentIndex yra 0, jei nurodyta innitialValue ir 1, jei innitialValue nenurodyta.**
4. **array – masyvas, kuriam kviečiame metodą reduce.**



JavaScript masyvų metodai
reduce( )

- **Lentelėje pavaizduota, kokius parametrus reduce metodas paduoda callback funkcijai ją kviesdamas pirmą kartą (pirmu prasisukimu), priklausomai nuo to, ar paduodama initialValue ar ne.**

```
let answer = array.reduce(function(totalResult,currValue,currIndex,array){},initialValue);
```

initialValue	previousValue	currentValue
passed	initialValue	array[0]
not passed	array[0]	array[1]

JavaScript masyvų metodai
reduce( )

```
//Declare an array of numbers
var array = [1,2,3,4,5];

//Declare a function that calculates and returns the product

function productArray(totalResult,currValue){
        return totalResult*currValue;
}

//Declare an answer variable to store the final result
//The reduce function callback is the productArray method
let answer = array.reduce(productArray);
console.log("Answer with no initial value")
console.log(answer);


console.log("Given an initial value of 2. The answer should be doubled")
let answer2 = array.reduce(productArray,2);
console.log(answer2);
```



JavaScript masyvų metodai
reduce( )

Apskaičiuosime už kokią sumą prekių yra krepšelyje.

```
let shoppingCart = [
  {
    product: 'phone',
    qty: 1,
    price: 500,
  },
  {
    product: 'Screen Protector',
    qty: 1,
    price: 10,
  },
  {
    product: 'Memory Card',
    qty: 2,
    price: 20,
  },
];
```

```
let total = shoppingCart.reduce(function (previousValue, currentValue) {
  return previousValue + currentValue.qty * currentValue.price;
}, 0);
```

